

Data Structures Home Work Set 4

Register No: 2025115058

Name: Sri Senthil Adavan K

1: Implement a Stack using Queue data structure.

Program:

```
#include <stdio.h>
```

```
int q[100], f = 0, r = 0;
```

```
void push(int x) {  
    int s = r - f;  
    q[r++] = x;  
    for (int i = 0; i < s; i++) {  
        q[r++] = q[f++];  
    }  
}
```

```
void pop() {  
    if (f < r) {  
        printf("%d ", q[f++]);  
    }  
}
```

```
int main() {  
    int n, val;  
    printf("Enter number of elements to push: ");  
    scanf("%d", &n);  
    for (int i = 0; i < n; i++) {  
        printf("Enter value: ");  
        scanf("%d", &val);  
        push(val);  
    }  
    printf("Popped elements: ");  
    while (f < r) {  
        pop();  
    }  
    return 0;  
}
```

Output:

```
Enter number of elements to push: 3
Enter value: 1
Enter value: 3
Enter value: 2
Popped elements: 2 3 1
```

2. Implement a Queue using Stack data structure.

Program:

```
#include <stdio.h>

int s1[100], t1 = -1, s2[100], t2 = -1;

void enqueue(int x) {
    s1[++t1] = x;
}

void dequeue() {
    if (t2 == -1) {
        while (t1 >= 0) {
            s2[++t2] = s1[t1--];
        }
    }
    if (t2 >= 0) {
        printf("%d ", s2[t2--]);
    }
}

int main() {
    int n, val;
    printf("Enter number of elements to enqueue: ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("Enter value: ");
        scanf("%d", &val);
        enqueue(val);
    }
    printf("Dequeued elements: ");
    for (int i = 0; i < n; i++) {
        dequeue();
    }
    return 0;
}
```

```
}
```

Output:

Enter number of elements to enqueue: 3

Enter value: 3

Enter value: 2

Enter value: 1

Dequeued elements: 3 2 1

3. Write a C program using stack data structure to implement managing the history of web pages visited.

Program:

```
#include <stdio.h>
```

```
char history[100];
```

```
int top = -1;
```

```
void visit(char page) {  
    history[++top] = page;  
    printf("Visited %c\n", page);  
}
```

```
void back() {  
    if (top > 0) {  
        top--;  
        printf("Back to %c\n", history[top]);  
    } else {  
        printf("No previous history\n");  
    }  
}
```

```
int main() {  
    int v, b;  
    char page;  
    printf("Enter number of pages to visit: ");  
    scanf("%d", &v);  
    for (int i = 0; i < v; i++) {  
        printf("Enter page initial: ");  
        scanf(" %c", &page);  
        visit(page);  
    }  
    printf("Enter number of times to go back: ");  
    scanf("%d", &b);  
    for (int i = 0; i < b; i++) {  
        back();  
    }  
    return 0;  
}
```

```
}
```

Output:

Enter number of pages to visit: 2

Enter page initial: 3

Visited 3

Enter page initial: 4

Visited 4

Enter number of times to go back: 3

Back to 3

No previous history

No previous history

4. Write a C program to sort a stack using temporary stack.

Program:

```
#include <stdio.h>
```

```
int s[100], t1 = -1, ts[100], t2 = -1;
```

```
void push(int x) {
```

```
    s[++t1] = x;
```

```
}
```

```
int pop() {
```

```
    return s[t1--];
```

```
}
```

```
int main() {
```

```
    int n, val;
```

```
    printf("Enter number of elements: ");
```

```
    scanf("%d", &n);
```

```

for (int i = 0; i < n; i++) {

    printf("Enter value: ");

    scanf("%d", &val);

    push(val);

}

while (t1 >= 0) {

    int tmp = pop();

    while (t2 >= 0 && ts[t2] > tmp) {

        push(ts[t2--]);

    }

    ts[++t2] = tmp;

}

printf("Sorted stack: ");

while (t2 >= 0) {

    printf("%d ", ts[t2--]);

}

return 0;

}

```

Output:

Enter number of elements: 3

Enter value: 23

Enter value: 67

Enter value: 23

Sorted stack: 67 27 23

5. The stock span problem where we have a series of daily price quotes for a stock... calculate the span of the stocks price for all days. Implement using stack.

Program:

Enter number of days: 3

Enter price for day 1: 200

Enter price for day 2: 250

Enter price for day 3: 300

Spans: 1 2 3

6. Given an array for each element find the nearest element to its right that has a higher frequency than the current element. If no such element exists set value to -1. Implement using stack.

Program:

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, a[100], f[1000] = {0}, s[100], top = -1, ans[100];
```

```
    printf("Enter number of elements: ");
```

```
    scanf("%d", &n);
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf("Enter value: ");
```

```
        scanf("%d", &a[i]);
```

```
        f[a[i]]++;
```

```
    }
```

```

for (int i = n - 1; i >= 0; i--) {

    while (top >= 0 && f[s[top]] <= f[a[i]]) {

        top--;

    }

    if (top == -1) {

        ans[i] = -1;

    } else {

        ans[i] = s[top];

    }

    s[++top] = a[i];

}

printf("Result: ");

for (int i = 0; i < n; i++) {

    printf("%d ", ans[i]);

}

return 0;

}

```

Output:

Enter number of elements: 3

Enter value: 3

Enter value: 4

Enter value: 5

Result: -1 -1 -1

7. Given a pattern containing only I's and D's. I for Increasing and D for decreasing. Device an algorithm using stack to print minimum number following that pattern. Digits from 1-9 and digits can't repeat.

Program:

```
#include <stdio.h>

#include <string.h>

int main() {

    char p[100];

    printf("Enter pattern (I and D): ");

    scanf("%s", p);

    int s[100], top = -1, n = strlen(p);

    printf("Minimum number: ");

    for (int i = 0; i <= n; i++) {

        s[++top] = i + 1;

        if (i == n || p[i] == 'I') {

            while (top >= 0) {

                printf("%d", s[top--]);

            }

        }

    }

    return 0;

}
```


Output:

Enter pattern (I and D): DIDIDIDIDIDI

Minimum number: 21436587109121113

8. Given an array of digits (values are from 0 to 9) find the minimum possible sum of two numbers formed from digits of the array... Use Queue for implementation.

Program:

```
#include <stdio.h>
```

```
int main() {  
    int n, a[100];  
    printf("Enter number of digits: ");  
    scanf("%d", &n);  
    for (int i = 0; i < n; i++) {  
        printf("Enter digit: ");  
        scanf("%d", &a[i]);  
    }  
    for (int i = 0; i < n; i++) {  
        for (int j = i + 1; j < n; j++) {  
            if (a[i] > a[j]) {  
                int t = a[i];  
                a[i] = a[j];  
                a[j] = t;  
            }  
        }  
    }  
}
```

```

    }

}

}

int q[100], f = 0, r = 0;

for (int i = 0; i < n; i++) {

    q[r++] = a[i];

}

int n1 = 0, n2 = 0;

while (f < r) {

    n1 = n1 * 10 + q[f++];

    if (f < r) {

        n2 = n2 * 10 + q[f++];

    }

}

printf("Minimum sum: %d", n1 + n2);

return 0;

}

```

Output:

Enter number of digits: 4

Enter digit: 9

Enter digit: 9

Enter digit: 9

Enter digit: 9

Minimum sum: 198

9. Given an array and a positive integer k, find the first negative integer for each window (contiguous subarray) of size k. If a window does not contain a negative integer, then print 0... Implement using queue.

Program:

```
#include <stdio.h>
```

```
int main() {  
  
    int n, k, a[100];  
  
    printf("Enter array size: ");  
  
    scanf("%d", &n);  
  
    for (int i = 0; i < n; i++) {  
  
        printf("Enter element: ");  
  
        scanf("%d", &a[i]);  
  
    }  
  
    printf("Enter window size k: ");  
  
    scanf("%d", &k);  
  
    int q[100], f = 0, r = 0;  
  
    printf("Result: ");  
  
    for (int i = 0; i < n; i++) {  
  
        if (a[i] < 0) {  
  
            q[r++] = i;  
  
        }  
  
    }  
  
}
```

```

    if (f < r && q[f] <= i - k) {

        f++;

    }

    if (i >= k - 1) {

        if (f < r) {

            printf("%d ", a[q[f]]);

        } else {

            printf("0 ");

        }

    }

}

return 0;

}

```

Output:

Enter array size: 4

Enter element: 2

Enter element: 5

Enter element: 9

Enter element: 1

Enter window size k: 2

Result: 0 0 0

10. Given a matrix of dimension $m \times n$ where each cell... 0: empty, 1: fresh, 2: rotten... find minimum time required so that all oranges become rotten... Use Queue for implementation.

Program:

```
#include <stdio.h>
```

```
int main() {
```

```
    int m, n, grid[100][100];
```

```
    printf("Enter number of rows: ");
```

```
    scanf("%d", &m);
```

```
    printf("Enter number of columns: ");
```

```
    scanf("%d", &n);
```

```
    int qx[1000], qy[1000], qt[1000], f = 0, r = 0, fresh = 0, time = 0;
```

```
    for (int i = 0; i < m; i++) {
```

```
        for (int j = 0; j < n; j++) {
```

```
            printf("Enter cell value (0, 1, or 2): ");
```

```
            scanf("%d", &grid[i][j]);
```

```
            if (grid[i][j] == 2) {
```

```
                qx[r] = i;
```

```
                qy[r] = j;
```

```
                qt[r++] = 0;
```

```
            } else {
```

```
                if (grid[i][j] == 1) {
```

```
                    fresh++;
```

```
                }
```

```

    }

}

}

int dx[] = {-1, 1, 0, 0}, dy[] = {0, 0, -1, 1};

while (f < r) {

    int x = qx[f], y = qy[f], t = qt[f++];

    time = t;

    for (int i = 0; i < 4; i++) {

        int nx = x + dx[i], ny = y + dy[i];

        if (nx >= 0 && nx < m && ny >= 0 && ny < n && grid[nx][ny] == 1) {

            grid[nx][ny] = 2;

            fresh--;

            qx[r] = nx;

            qy[r] = ny;

            qt[r++] = t + 1;

        }

    }

}

printf("Minimum time required: ");

if (fresh > 0) {

    printf("-1");

} else {

    printf("%d", time);

}

return 0;

```

}

Output:

Enter number of rows: 3

Enter number of columns: 2

Enter cell value (0, 1, or 2): 2

Enter cell value (0, 1, or 2): 2

Enter cell value (0, 1, or 2): 0

Enter cell value (0, 1, or 2): 1

Enter cell value (0, 1, or 2): 0

Enter cell value (0, 1, or 2): 1

Minimum time required: 2